COMPUTER PROGRAMMING

Assignment No. 2 · Functions | 1D Arrays | 2D Arrays | String Arrays

Student Name	Talha Ali
Registration No.	BF25NWELE0728
Section	Electrical Power
Semester	2nd Semester
Subject	Computer Programming
Submitted To	Engr. M. Rizwan
Submission Date	4th May, 2026
University	UET Peshawar — Nowshera Campus

Q1. 1D Array — Maximum, Minimum & Average

```
#include <iostream>
using namespace std;

int main() {
    int arr[10], sum = 0, max, min;

    // input 10 integers
    cout << "Enter 10 integers: ";
    for (int i = 0; i < 10; i++)
        cin >> arr[i];

    max = min = arr[0];  // start comparison from first element

    for (int i = 0; i < 10; i++) {
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
        sum += arr[i];
    }

    // display results
    cout << "Max: "     << max          << endl;
    cout << "Min: "     << min          << endl;
    cout << "Average: " << sum / 10.0  << endl;

    return 0;
}
```

Q2. 1D Array — Reverse Using a Function

```
#include <iostream>
using namespace std;

// reverses array in-place using two-pointer swap
void reverseArray(int arr[], int n) {
    int temp;
    for (int i = 0; i < n / 2; i++) {
        temp           = arr[i];
        arr[i]         = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
```

```
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;

    cout << "Before: "; printArray(arr, n);
    reverseArray(arr, n);                    // call reverse function
    cout << "After:  "; printArray(arr, n);

    return 0;
}
```

Q3. 2D Array — 3×3 Matrix Addition

```
#include <iostream>
using namespace std;

int main() {
    int A[3][3], B[3][3], C[3][3];

    // input first matrix
    cout << "Enter elements of Matrix A:" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cin >> A[i][j];

    // input second matrix
    cout << "Enter elements of Matrix B:" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cin >> B[i][j];

    // add corresponding elements
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            C[i][j] = A[i][j] + B[i][j];

    // display result matrix
    cout << "Result Matrix:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            cout << C[i][j] << " ";
        cout << endl;
    }

    return 0;
}
```

Q4. String Array — Alphabetical Sorting

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string names[5], temp;

    // input 5 names
    cout << "Enter 5 names: ";
    for (int i = 0; i < 5; i++)
        cin >> names[i];
```

```
    // selection sort — swap if out of alphabetical order
    for (int i = 0; i < 5; i++)
        for (int j = i + 1; j < 5; j++)
            if (names[i] > names[j]) {
                temp     = names[i];
                names[i] = names[j];
                names[j] = temp;
            }

    // display sorted names
    cout << "Sorted: ";
    for (int i = 0; i < 5; i++)
        cout << names[i] << " ";

    return 0;
}
```

Q5. Power System Load Analysis

```
#include <iostream>
using namespace std;

// returns sum of all hourly loads
float totalLoad(float loads[], int n) {
    float sum = 0;
    for (int i = 0; i < n; i++) sum += loads[i];
    return sum;
}

// returns highest load value
float peakLoad(float loads[], int n) {
    float peak = loads[0];
    for (int i = 1; i < n; i++)
        if (loads[i] > peak) peak = loads[i];
    return peak;
}

int main() {
    float loads[24];

    // input 24-hour load data
    cout << "Enter load demand for 24 hours (MW):" << endl;
    for (int i = 0; i < 24; i++) {
        cout << "Hour " << i + 1 << ": ";
        cin >> loads[i];
    }

    cout << "Total Load : " << totalLoad(loads, 24) << " MW" << endl;
    cout << "Peak Load  : " << peakLoad(loads, 24)  << " MW" << endl;

    return 0;
}
```

Q6. Student Result System

```
#include <iostream>
#include <string>
using namespace std;

const int N = 5;

// finds and prints student with highest marks
void findTopper(string names[], int marks[]) {
    int top = 0;
    for (int i = 1; i < N; i++)
        if (marks[i] > marks[top]) top = i;
    cout << "Topper: " << names[top] << " (" << marks[top] << ")" << endl;
}
```

```cpp
// prints pass/fail for each student (pass >= 50)
void showResult(string names[], int marks[]) {
    for (int i = 0; i < N; i++)
        cout << names[i] << ": " << (marks[i] >= 50 ? "Pass" : "Fail") << endl;
}

// searches student by name and prints marks
void searchStudent(string names[], int marks[]) {
    string name;
    cout << "Enter name to search: "; cin >> name;
    for (int i = 0; i < N; i++)
        if (names[i] == name) {
            cout << "Found: " << names[i] << " - Marks: " << marks[i] << endl;
            return;
        }
    cout << "Not found." << endl;
}

int main() {
    string names[N];
    int marks[N];

    for (int i = 0; i < N; i++) {
        cout << "Name: ";  cin >> names[i];
        cout << "Marks: "; cin >> marks[i];
    }

    findTopper(names, marks);
    showResult(names, marks);
    searchStudent(names, marks);

    return 0;
}
```

Q7. Renewable Energy Monitoring System

```cpp
#include <iostream>
using namespace std;

// returns total energy generated
float calcTotal(float e[], int n) {
    float sum = 0;
    for (int i = 0; i < n; i++) sum += e[i];
    return sum;
}

// returns index of hour with max generation
int findPeakHour(float e[], int n) {
    int peak = 0;
    for (int i = 1; i < n; i++)
        if (e[i] > e[peak]) peak = i;
    return peak;
}

// returns average energy per hour
float calcAverage(float total, int n) {
    return total / n;
}

int main() {
    float energy[24];

    // input hourly energy data
    cout << "Enter energy generated each hour (kWh):" << endl;
    for (int i = 0; i < 24; i++) {
        cout << "Hour " << i + 1 << ": ";
        cin >> energy[i];
    }

    float total = calcTotal(energy, 24);
```

```
    int   peak  = findPeakHour(energy, 24);

    cout << "Total Energy : " << total                    << " kWh" << endl;
    cout << "Average      : " << calcAverage(total, 24)   << " kWh" << endl;
    cout << "Peak Hour    : Hour " << peak + 1 << " (" << energy[peak] << " kWh)" << endl;

    return 0;
}
```

Q8. Battery Health Monitoring

```
#include <iostream>
using namespace std;

// counts readings outside safe range [3.0V, 4.2V] and warns
void checkSafety(float v[], int n) {
    int below = 0, above = 0;
    for (int i = 0; i < n; i++) {
        if (v[i] < 3.0) below++;
        else if (v[i] > 4.2) above++;
    }
    cout << "Below 3.0V : " << below << endl;
    cout << "Above 4.2V : " << above << endl;
    if (below > 0 || above > 0)
        cout << "WARNING: Unsafe readings detected!" << endl;
}

// finds and prints minimum and maximum voltage
void findMinMax(float v[], int n) {
    float mn = v[0], mx = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] < mn) mn = v[i];
        if (v[i] > mx) mx = v[i];
    }
    cout << "Min Voltage : " << mn << " V" << endl;
    cout << "Max Voltage : " << mx << " V" << endl;
}

int main() {
    float voltages[20];

    // input 20 voltage readings
    cout << "Enter 20 voltage readings (V):" << endl;
    for (int i = 0; i < 20; i++) {
        cout << "Reading " << i + 1 << ": ";
        cin >> voltages[i];
    }

    checkSafety(voltages, 20);
    findMinMax(voltages, 20);

    return 0;
}
```

Q9. Smart Grid Load Distribution

```
#include <iostream>
using namespace std;

const int R = 4, D = 7;  // 4 regions, 7 days

// prints total consumption for each region
void regionTotals(float g[R][D]) {
    for (int r = 0; r < R; r++) {
        float sum = 0;
        for (int d = 0; d < D; d++) sum += g[r][d];
        cout << "Region " << r + 1 << " Total: " << sum << " MW" << endl;
    }
}
```

```
// finds day with highest combined demand across all regions
void peakDay(float g[R][D]) {
    float daySum[D] = {0};
    for (int d = 0; d < D; d++)
        for (int r = 0; r < R; r++)
            daySum[d] += g[r][d];
    int peak = 0;
    for (int i = 1; i < D; i++)
        if (daySum[i] > daySum[peak]) peak = i;
    cout << "Peak Day: Day " << peak + 1 << " (" << daySum[peak] << " MW)" << endl;
}

// returns average over all regions and days
float overallAvg(float g[R][D]) {
    float sum = 0;
    for (int r = 0; r < R; r++)
        for (int d = 0; d < D; d++)
            sum += g[r][d];
    return sum / (R * D);
}

int main() {
    float grid[R][D];

    cout << "Enter consumption for 4 regions over 7 days (MW):" << endl;
    for (int r = 0; r < R; r++)
        for (int d = 0; d < D; d++) {
            cout << "Region " << r+1 << " Day " << d+1 << ": ";
            cin >> grid[r][d];
        }

    regionTotals(grid);
    peakDay(grid);
    cout << "Overall Average: " << overallAvg(grid) << " MW" << endl;

    return 0;
}
```

Q10. Classroom Attendance System

```
#include <iostream>
using namespace std;

const int S = 5, D = 7;  // 5 students, 7 days

// prints total days present for each student
void studentAttendance(int a[S][D]) {
    for (int s = 0; s < S; s++) {
        int total = 0;
        for (int d = 0; d < D; d++) total += a[s][d];
        cout << "Student " << s + 1 << ": " << total << " days present" << endl;
    }
}

// finds student with highest attendance
void bestStudent(int a[S][D]) {
    int best = 0, bestCount = 0;
    for (int s = 0; s < S; s++) {
        int count = 0;
        for (int d = 0; d < D; d++) count += a[s][d];
        if (count > bestCount) { bestCount = count; best = s; }
    }
    cout << "Highest Attendance: Student " << best + 1 << " (" << bestCount << " days)" << endl;
}

// calculates overall class attendance percentage
void classPercentage(int a[S][D]) {
    int present = 0;
    for (int s = 0; s < S; s++)
        for (int d = 0; d < D; d++)
```

```
            present += a[s][d];
    cout << "Class Attendance: " << (present * 100.0) / (S * D) << "%" << endl;
}

int main() {
    int att[S][D];

    // input attendance: 1 = present, 0 = absent
    cout << "Enter attendance (1=Present, 0=Absent):" << endl;
    for (int s = 0; s < S; s++) {
        cout << "Student " << s + 1 << " (7 days): ";
        for (int d = 0; d < D; d++) cin >> att[s][d];
    }

    studentAttendance(att);
    bestStudent(att);
    classPercentage(att);

    return 0;
}
```